

SE-Lab4-Chat History

09/30/2026 17:58:56

<https://chatgpt.com/c/6abcae9a-571c-83ee-b982-6e605aec2d28>

I am working on a Pygame Simon-style memory game. Please review th...
The bug is exactly here: `self.sequence += self.sequence + [new_color]` ...
I understand the fix. I have applied the suggested change and tested th...
Great. The sequence duplication bug is fixed and verified. Send me the...
Now implement the second task only: dynamic playback acceleration....
Approach Use the player's score to calculate shorter durations whenever...
I applied the dynamic playback changes and tested several rounds. The...
Sure. Send me the sound-effects requirements or the relevant current...
Implement the third task: sound effects for the four colored buttons. Us...
A simple approach is to synthesize short sine-wave tones once during...
I implemented and tested the color-specific sound effects. The...
Absolutely. Please send the countdown timer requirements (for exampl...
Implement the fourth and final task: a round countdown timer. During...
A clean way to add this is to give each `PLAYER_TURN` a fixed time limit,...
I have now implemented all four tasks: 1. Fixed the sequence duplicatio...
Your final implementation satisfies the listed requirements. I found no...



I am working on a Pygame Simon-style memory game. Please review the following `game\_engine.py` code.

The game has an intentional sequence duplication bug in `start\_next\_round()`. Each round should append exactly one new random color to the existing sequence, but the current implementation duplicates the existing sequence.

Please:

1. Identify the exact bug.
2. Explain briefly why it causes exponential sequence growth.
3. Suggest the minimal code change to fix only this bug.
4. Do not implement any of the other tasks yet.

Here is the relevant code:

```
import random
import pygame
from game.color_button import ColorButton

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        pad_size = 130
        gap = 24
        start_x = width // 2 - pad_size - (gap // 2)
        start_y = 150

        self.buttons = [
            ColorButton(0, pygame.Rect(start_x, start_y,
                                        pad_size, pad_size), (110, 20, 20), (255, 50, 50)),
            # Red
```

```

        ColorButton(1, pygame.Rect(start_x + pad_size
+ gap, start_y, pad_size, pad_size), (15, 60, 150), (40,
170, 255)), # Blue
        ColorButton(2, pygame.Rect(start_x, start_y +
pad_size + gap, pad_size, pad_size), (15, 100, 30), (50,
255, 90)), # Green
        ColorButton(3, pygame.Rect(start_x + pad_size
+ gap, start_y + pad_size + gap, pad_size, pad_size),
(140, 110, 10), (255, 235, 40)), # Yellow
    ]

    self.sequence = []
    self.player_input = []
    self.score = 0

    self.state = "WATCH"
    self.showing_step = 0
    self.step_start_time = 0
    self.flash_duration = 450
    self.pause_duration = 200
    self.is_flashing = False

    self.player_lit_button = None
    self.player_lit_start = 0
    self.player_flash_duration = 150

    self.font_title = pygame.font.SysFont(None, 40)
    self.font_medium = pygame.font.SysFont(None,
28)

    self.start_next_round()

    def start_next_round(self):
        new_color = random.randint(0, 3)

        # BUG SYMPTOM:
        # Sequence duplicates previous steps and grows
        exponentially each round.
        self.sequence += self.sequence + [new_color]

        self.player_input.clear()

```

```

self.state = "WATCH"
self.showing_step = 0
self.step_start_time = pygame.time.get_ticks()
self.is_flashing = True
self.buttons[self.sequence[0]].is_lit = True

def update(self):
    now = pygame.time.get_ticks()

    if self.player_lit_button is not None:
        if now - self.player_lit_start >=
self.player_flash_duration:
            self.player_lit_button.is_lit = False
            self.player_lit_button = None

    if self.state == "WATCH":
        current_btn_id =
self.sequence[self.showing_step]

        if self.is_flashing:
            if now - self.step_start_time >=
self.flash_duration:
                self.buttons[current_btn_id].is_lit = False
                self.is_flashing = False
                self.step_start_time = now
            else:
                if now - self.step_start_time >=
self.pause_duration:
                    self.showing_step += 1
                    if self.showing_step < len(self.sequence):
                        next_id =
self.sequence[self.showing_step]
                        self.buttons[next_id].is_lit = True
                        self.is_flashing = True
                        self.step_start_time = now
                    else:
                        self.state = "PLAYER_TURN"

def handle_event(self, event):
    if self.state == "GAME_OVER":
        if event.type == pygame.KEYDOWN and

```

```

event.key == pygame.K_r:
    self.reset()
    return

    if self.state == "PLAYER_TURN" and event.type
    == pygame.MOUSEBUTTONDOWN and event.button
    == 1:
        for btn in self.buttons:
            if btn.contains(event.pos):
                btn.is_lit = True
                self.player_lit_button = btn
                self.player_lit_start =
pygame.time.get_ticks()

                self.register_player_click(btn.color_id)
                break

def register_player_click(self, color_id):
    self.player_input.append(color_id)
    current_idx = len(self.player_input) - 1

    if self.player_input[current_idx] !=
self.sequence[current_idx]:
        self.state = "GAME_OVER"
        return

    if len(self.player_input) == len(self.sequence):
        self.score += 1
        self.start_next_round()

def reset(self):
    self.sequence.clear()
    self.player_input.clear()
    self.score = 0
    for btn in self.buttons:
        btn.is_lit = False
    self.player_lit_button = None
    self.start_next_round()

def render(self, screen):
    screen.fill((22, 24, 30))

```

```

        title_surf = self.font_title.render("Memory Pattern
Arena", True, (245, 245, 245))
        screen.blit(title_surf, (self.width // 2 -
title_surf.get_width() // 2, 20))

        score_surf = self.font_medium.render(f"Score:
{self.score}", True, (255, 220, 80))
        screen.blit(score_surf, (self.width // 2 -
score_surf.get_width() // 2, 60))

        status_text = "Watch the pattern..." if self.state
== "WATCH" else "Your turn: Click the pattern!"
        status_color = (190, 195, 205) if self.state ==
"WATCH" else (80, 240, 130)
        status_surf = self.font_medium.render(status_text,
True, status_color)
        screen.blit(status_surf, (self.width // 2 -
status_surf.get_width() // 2, 95))

        for btn in self.buttons:
            btn.render(screen)

        if self.state == "GAME_OVER":
            overlay = pygame.Surface((self.width,
self.height), pygame.SRCALPHA)
            overlay.fill((0, 0, 0, 200))
            screen.blit(overlay, (0, 0))

            over_surf = self.font_title.render("WRONG
PATTERN! GAME OVER", True, (240, 70, 70))
            screen.blit(over_surf, (self.width // 2 -
over_surf.get_width() // 2, self.height // 2 - 40))

            final_score_surf =
self.font_medium.render(f"Final Score: {self.score}",
True, (255, 255, 255))
            screen.blit(final_score_surf, (self.width // 2 -
final_score_surf.get_width() // 2, self.height // 2 + 10))

            restart_surf = self.font_medium.render("Press

```

```
[R] to Play Again", True, (200, 200, 200))
    screen.blit(restart_surf, (self.width // 2 -
restart_surf.get_width() // 2, self.height // 2 + 50))
```

9/30/2026, 12:09:52 PM



The bug is exactly here:

```
self.sequence += self.sequence + [new_color]
```

1. Exact bug

`self.sequence` is being added to itself before the new color:

```
self.sequence + [new_color]
```

and then that entire result is appended to `self.sequence` again with `+=`.

So if:

```
sequence = [1, 2]
new_color = 3
```

the expression becomes:

```
self.sequence += [1, 2, 3]
```

resulting in:

```
[1, 2, 1, 2, 3]
```

2. Why it grows exponentially

Each round roughly **doubles the existing sequence**, then adds one new color.

The lengths therefore go:

```
Round 1: 1
Round 2: 3
```

```
Round 3: 7
Round 4: 15
Round 5: 31
...
```

So the sequence length follows approximately:

$2^n - 1$, rather than increasing by exactly one each round.

3. Minimal fix

Change only this line:

```
self.sequence += self.sequence + [new_color]
```

to:

```
self.sequence.append(new_color)
```

So `start_next_round()` becomes:

```
def start_next_round(self):
    new_color = random.randint(0, 3)

    self.sequence.append(new_color)

    self.player_input.clear()
    self.state = "WATCH"
    self.showing_step = 0
    self.step_start_time = pygame.time.get_ticks()
    self.is_flashing = True
    self.buttons[self.sequence[0]].is_lit = True
```

Now the sequence grows correctly:

```
Round 1: [2]
Round 2: [2, 0]
Round 3: [2, 0, 3]
Round 4: [2, 0, 3, 1]
```

No other code needs to be changed for this bug.



I understand the fix. I have applied the suggested change and tested the game. The sequence now grows by exactly one new color each round.

Let's move to the next task.

9/30/2026, 12:10:14 PM



Great. The sequence duplication bug is fixed and verified.

Send me the **next task/bug requirement**, and I'll focus only on that change without modifying unrelated behavior.

9/30/2026, 12:10:15 PM · gpt-5-6



Now implement the second task only: dynamic playback acceleration.

Currently the game uses:

- flash duration = 450 ms
- pause duration = 200 ms

Modify the game so that both durations gradually decrease as the player's score increases. The minimum values must be:

- flash duration: 180 ms
- pause duration: 80 ms

Please:

1. Explain your approach briefly.
2. Show the specific code changes.
3. Keep the rest of the game behavior unchanged.
4. Do not implement the sound effects or countdown timer yet.

9/30/2026, 12:10:29 PM



Approach

Use the **player's score** to calculate shorter durations whenever a new round starts. The durations decrease gradually but are capped at the required minimums.

A simple formula is:

- Flash: $450 - \text{score} \times 15$, minimum 180
- Pause: $200 - \text{score} \times 8$, minimum 80

This keeps the acceleration gradual and requires no changes to the playback state machine.

1. Keep the existing defaults

In `__init__`, keep:

```
self.flash_duration = 450
self.pause_duration = 200
```

2. Update `start_next_round()`

Add the duration calculation at the beginning of `start_next_round()`:

```
def start_next_round(self):
    new_color = random.randint(0, 3)

    self.sequence.append(new_color)

    # Gradually accelerate playback as the score increases.
    self.flash_duration = max(180, 450 - self.score * 15)
    self.pause_duration = max(80, 200 - self.score * 8)

    self.player_input.clear()
    self.state = "WATCH"
    self.showing_step = 0
    self.step_start_time = pygame.time.get_ticks()
    self.is_flashing = True
    self.buttons[self.sequence[0]].is_lit = True
```

What this does

Score	Flash	Pause
0	450 ms	200 ms
1	435 ms	192 ms
5	375 ms	160 ms
10	300 ms	120 ms
18	180 ms	80 ms
25+	180 ms	80 ms

Once either duration reaches its minimum, it **stays there**.

Everything else—including sequence generation, player input, scoring, game-over behavior, and rendering—remains unchanged. No sound effects or countdown timer are added.

9/30/2026, 12:10:30 PM · gpt-5-6



I applied the dynamic playback changes and tested several rounds. The playback becomes faster as the score increases while respecting the minimum duration limits.

Let's move to the sound-effects task.

9/30/2026, 12:11:54 PM



Sure. Send me the **sound-effects requirements** or the relevant current code/files, and I'll implement **only the sound-effects task**, leaving the sequence logic and dynamic playback acceleration unchanged.

9/30/2026, 12:11:55 PM · gpt-5-6



Implement the third task: sound effects for the four colored buttons.

Use Pygame audio or synthesized tones with these frequencies:

- Red: 261 Hz
- Blue: 329 Hz
- Green: 392 Hz
- Yellow: 523 Hz

Requirements:

1. Play the corresponding tone whenever a button flashes during WATCH mode.
2. Play the corresponding tone when the player clicks a button during PLAYER_TURN.
3. Keep the existing visual behavior unchanged.
4. Use a simple implementation suitable for this Pygame project.
5. Do not implement the countdown timer yet.

Please explain the changes and show the relevant code.

9/30/2026, 12:12:07 PM



A simple approach is to **synthesize short sine-wave tones once during initialization**, then reuse those `pygame.mixer.Sound` objects whenever a button is flashed or clicked. This avoids generating audio repeatedly during gameplay.

1. Add the audio initialization

In `__init__`, after `self.buttons` is created, add:

```
pygame.mixer.init()
```

```

self.tones = {
    0: self.create_tone(261), # Red
    1: self.create_tone(329), # Blue
    2: self.create_tone(392), # Green
    3: self.create_tone(523), # Yellow
}

```

Then add this method to `GameEngine`:

```

def create_tone(self, frequency, duration=0.15):
    sample_rate = 44100
    samples = int(sample_rate * duration)

    buffer = bytearray()

    for i in range(samples):
        value = int(
            32767 * 0.25 *
            math.sin(2 * math.pi * frequency * i / sample_rate)
        )

        buffer.extend(value.to_bytes(2, byteorder="little", signed=True))

    return pygame.mixer.Sound(buffer=bytes(buffer))

```

You'll also need:

```
import math
```

at the top.

2. Play the tone during WATCH mode

You currently light the first button here:

```
self.buttons[self.sequence[0]].is_lit = True
```

Change it to:

```
self.buttons[self.sequence[0]].is_lit = True
```

```
self.tones[self.sequence[0]].play()
```

And when moving to the next button in `update()`:

```
next_id = self.sequence[self.showing_step]
self.buttons[next_id].is_lit = True
self.tones[next_id].play()
self.is_flashing = True
self.step_start_time = now
```

3. Play the tone when the player clicks

In `handle_event()`, you currently have:

```
btn.is_lit = True
self.player_lit_button = btn
self.player_lit_start = pygame.time.get_ticks()

self.register_player_click(btn.color_id)
```

Add the sound:

```
btn.is_lit = True
self.tones[btn.color_id].play()

self.player_lit_button = btn
self.player_lit_start = pygame.time.get_ticks()

self.register_player_click(btn.color_id)
```

4. One additional import

Your imports become:

```
import random
import math
import pygame
from game.color_button import ColorButton
```

The resulting mapping is:

Red	→ 261 Hz
Blue	→ 329 Hz
Green	→ 392 Hz
Yellow	→ 523 Hz

This leaves your **visual flashing, sequence logic, scoring, dynamic playback acceleration, and game-over behavior unchanged**. The countdown timer is not implemented.

9/30/2026, 12:12:09 PM · gpt-5-6



I implemented and tested the color-specific sound effects. The appropriate tone plays during the sequence playback and when the player clicks a color.

Let's implement the final task: the countdown timer.

9/30/2026, 12:12:21 PM



Absolutely. Please send the **countdown timer requirements** (for example, countdown length, when it should appear, and what should happen when it reaches zero).

I'll implement **only the timer**, keeping your existing sequence logic, dynamic acceleration, sound effects, and visual behavior unchanged.

9/30/2026, 12:12:23 PM · gpt-5-6



Implement the fourth and final task: a round countdown timer.

During `PLAYER_TURN`:

1. Display a visible countdown/timer bar in the HUD.
2. Give the player a limited amount of time to complete the current sequence.
3. Update the timer as time passes.
4. If the timer reaches zero before the player completes the sequence, transition the state to `GAME_OVER`.
5. Keep the existing incorrect-input Game Over behavior unchanged.
6. Make sure the timer resets appropriately when a new round begins or the game is restarted.

Please explain the implementation and show the relevant code changes.

9/30/2026, 12:14:52 PM



A clean way to add this is to give each **PLAYER_TURN** a fixed time limit, track when that turn started, and draw a shrinking bar in the HUD.

I'll use **10 seconds per player turn**. The timer only runs during `PLAYER_TURN`; `WATCH` playback is unaffected.

1. Add timer variables in `__init__`

After your existing player-flash variables, add:

```
self.turn_time_limit = 10000 # 10 seconds, in milliseconds
self.turn_start_time = 0
```

2. Start the timer when `WATCH` finishes

In `update()`, you currently have:

```
else:
    self.state = "PLAYER_TURN"
```

Change it to:

```
else:
    self.state = "PLAYER_TURN"
    self.turn_start_time = now
```

This means the player gets the full 10 seconds **after the sequence has finished playing**.

3. Check the timer during PLAYER_TURN

In `update()`, after the existing `WATCH` block, add:

```
if self.state == "PLAYER_TURN":
    elapsed = now - self.turn_start_time

    if elapsed >= self.turn_time_limit:
        self.state = "GAME_OVER"
    else:
        remaining = self.turn_time_limit - elapsed
```

The important part is the transition:

```
if elapsed >= self.turn_time_limit:
    self.state = "GAME_OVER"
```

Your existing incorrect-input behavior remains untouched:

```
if self.player_input[current_idx] != self.sequence[current_idx]:
    self.state = "GAME_OVER"
    return
```

4. Reset the timer for every new round

In `start_next_round()`, reset the timer state:

```
self.turn_start_time = 0
```

So the relevant part becomes:

```
def start_next_round(self):
    new_color = random.randint(0, 3)

    self.sequence.append(new_color)

    self.flash_duration = max(180, 450 - self.score * 15)
    self.pause_duration = max(80, 200 - self.score * 8)

    self.player_input.clear()
    self.state = "WATCH"
    self.showing_step = 0
    self.step_start_time = pygame.time.get_ticks()
    self.turn_start_time = 0
    self.is_flashing = True
    self.buttons[self.sequence[0]].is_lit = True
    self.tones[self.sequence[0]].play()
```

When the player eventually finishes WATCH, `turn_start_time` is set to the current time.

5. Draw the countdown bar

In `render()`, after your status text, add:

```
if self.state == "PLAYER_TURN":
    now = pygame.time.get_ticks()
    elapsed = now - self.turn_start_time
    remaining_ratio = max(
        0,
        1 - elapsed / self.turn_time_limit
    )

    bar_width = 300
    bar_height = 18
    bar_x = self.width // 2 - bar_width // 2
    bar_y = 125
```

```

# Background
pygame.draw.rect(
    screen,
    (60, 60, 65),
    (bar_x, bar_y, bar_width, bar_height)
)

# Remaining time
pygame.draw.rect(
    screen,
    (80, 220, 120),
    (
        bar_x,
        bar_y,
        int(bar_width * remaining_ratio),
        bar_height
    )
)

# Optional timer text
remaining_seconds = max(
    0,
    (self.turn_time_limit - elapsed) / 1000
)

timer_surf = self.font_medium.render(
    f"Time: {remaining_seconds:.1f}s",
    True,
    (245, 245, 245)
)

screen.blit(
    timer_surf,
    (
        self.width // 2 - timer_surf.get_width() // 2,
        bar_y + 25
    )
)

```

This gives the player both a **visual shrinking bar** and a numerical countdown.

6. Reset behavior

Your existing `reset()` already calls:

```
self.start_next_round()
```

Since `start_next_round()` now resets:

```
self.turn_start_time = 0
```

the timer is automatically reset when restarting the game.

Resulting flow

```
WATCH
  ↓
Sequence finishes
  ↓
PLAYER_TURN
  ↓
10-second timer starts
  ↓
Player completes sequence → next round / timer reset
  ↓
Timer reaches 0
  ↓
GAME_OVER
```

Incorrect input still immediately produces `GAME_OVER` exactly as before.

The timer does **not** affect WATCH playback, sequence generation, sound effects, scoring, or your dynamic playback acceleration.

9/30/2026, 12:14:54 PM · gpt-5-6



I have now implemented all four tasks:

1. Fixed the sequence duplication bug.
2. Added dynamic playback acceleration with minimum flash/pause limits.
3. Added color-specific sound effects.
4. Added a PLAYER_TURN countdown timer that leads to GAME_OVER when time expires.

Please review the final implementation against the following requirements and identify anything that is missing or incorrect. Do not add unrelated features.

```
import random
```

```
import math
```

```
import pygame
```

```
from game.color_button import ColorButton
```

```
class GameEngine:
```

```
    def __init__(self, width, height):
```

```
        self.width = width
```

```
        self.height = height
```

```
        pad_size = 130
```

```
        gap = 24
```

```
        start_x = width // 2 - pad_size - (gap // 2)
```

```
        start_y = 150
```

```
        self.buttons = [
```

```
            ColorButton(
```

```
                0,
```

```
                pygame.Rect(start_x, start_y, pad_size,
```

```
pad_size),
```

```
                (110, 20, 20),
```

```
                (255, 50, 50)
```

```
            ), # Red
```

```
            ColorButton(
```

```
                1,
```

```
                pygame.Rect(
```

```

        start_x + pad_size + gap,
        start_y,
        pad_size,
        pad_size
    ),
    (15, 60, 150),
    (40, 170, 255)
), # Blue
ColorButton(
    2,
    pygame.Rect(
        start_x,
        start_y + pad_size + gap,
        pad_size,
        pad_size
    ),
    (15, 100, 30),
    (50, 255, 90)
), # Green
ColorButton(
    3,
    pygame.Rect(
        start_x + pad_size + gap,
        start_y + pad_size + gap,
        pad_size,
        pad_size
    ),
    (140, 110, 10),
    (255, 235, 40)
), # Yellow
]

self.sequence = []
self.player_input = []
self.score = 0

self.state = "WATCH"
self.showing_step = 0
self.step_start_time = 0

# Dynamic playback speed

```

```

self.flash_duration = 450
self.pause_duration = 200
self.is_flashing = False

self.player_lit_button = None
self.player_lit_start = 0
self.player_flash_duration = 150

# Player countdown
self.player_turn_duration = 5000
self.player_turn_start = 0

# Sound effects
self.sound_frequencies = {
    0: 261, # Red
    1: 329, # Blue
    2: 392, # Green
    3: 523  # Yellow
}

self.sounds = {}
self._create_sounds()

self.font_title = pygame.font.SysFont(None, 40)
self.font_medium = pygame.font.SysFont(None,
28)

self.start_next_round()

def _create_sounds(self):
    """
    Create simple tones for each color.

    pygame.mixer must already be initialized by the
    main program.
    """
    try:
        sample_rate = 44100

        for color_id, frequency in
self.sound_frequencies.items():

```

```

        duration = 0.15
        samples = int(sample_rate * duration)

        sound_buffer = bytearray()

        for i in range(samples):
            value = int(
                32767
                * 0.25
                * math.sin(
                    2 * math.pi * frequency * i /
sample_rate
                )
            )

            sound_buffer += int(value).to_bytes(
                2,
                byteorder="little",
                signed=True
            )

        sound =
pygame.mixer.Sound(buffer=bytes(sound_buffer))
        self.sounds[color_id] = sound

    except (pygame.error, ValueError):
        self.sounds = {}

    def play_color_sound(self, color_id):
        sound = self.sounds.get(color_id)

        if sound is not None:
            sound.play()

    def update_speed(self):
        """
        Make playback faster as the score increases.

        Minimum limits:
        flash >= 180 ms
        pause >= 80 ms

```

```

"""
self.flash_duration = max(
    180,
    450 - self.score * 20
)

self.pause_duration = max(
    80,
    200 - self.score * 10
)

def start_next_round(self):
    new_color = random.randint(0, 3)

    # Add exactly one new color to the sequence.
    self.sequence.append(new_color)

    self.update_speed()

    self.player_input.clear()
    self.state = "WATCH"
    self.showing_step = 0
    self.step_start_time = pygame.time.get_ticks()
    self.is_flashing = True

    first_button = self.buttons[self.sequence[0]]
    first_button.is_lit = True
    self.play_color_sound(self.sequence[0])

def start_player_turn(self):
    self.state = "PLAYER_TURN"
    self.player_turn_start = pygame.time.get_ticks()

def update(self):
    now = pygame.time.get_ticks()

    # Turn off player button after its flash duration.
    if self.player_lit_button is not None:
        if now - self.player_lit_start >=
self.player_flash_duration:
        self.player_lit_button.is_lit = False

```

```

        self.player_lit_button = None

        if self.state == "WATCH":
            current_btn_id =
self.sequence[self.showing_step]

            if self.is_flashing:
                if now - self.step_start_time >=
self.flash_duration:
                    self.buttons[current_btn_id].is_lit = False
                    self.is_flashing = False
                    self.step_start_time = now

            else:
                if now - self.step_start_time >=
self.pause_duration:
                    self.showing_step += 1

                    if self.showing_step < len(self.sequence):
                        next_id =
self.sequence[self.showing_step]

                        self.buttons[next_id].is_lit = True
                        self.play_color_sound(next_id)

                        self.is_flashing = True
                        self.step_start_time = now

                    else:
                        self.start_player_turn()

        elif self.state == "PLAYER_TURN":
            elapsed = now - self.player_turn_start

            if elapsed >= self.player_turn_duration:
                self.state = "GAME_OVER"

        def handle_event(self, event):
            if self.state == "GAME_OVER":
                if event.type == pygame.KEYDOWN and
event.key == pygame.K_r:

```

```

        self.reset()
        return

    if (
        self.state == "PLAYER_TURN"
        and event.type ==
pygame.MOUSEBUTTONDOWN
        and event.button == 1
    ):
        for btn in self.buttons:
            if btn.contains(event.pos):
                btn.is_lit = True
                self.player_lit_button = btn
                self.player_lit_start =
pygame.time.get_ticks()

                self.play_color_sound(btn.color_id)
                self.register_player_click(btn.color_id)
                break

def register_player_click(self, color_id):
    self.player_input.append(color_id)

    current_idx = len(self.player_input) - 1

    if self.player_input[current_idx] !=
self.sequence[current_idx]:
        self.state = "GAME_OVER"
        return

    if len(self.player_input) == len(self.sequence):
        self.score += 1
        self.start_next_round()

def reset(self):
    self.sequence.clear()
    self.player_input.clear()
    self.score = 0

    for btn in self.buttons:
        btn.is_lit = False

```

```

        self.player_lit_button = None

        self.start_next_round()

    def render(self, screen):
        screen.fill((22, 24, 30))

        title_surf = self.font_title.render(
            "Memory Pattern Arena",
            True,
            (245, 245, 245)
        )
        screen.blit(
            title_surf,
            (
                self.width // 2 - title_surf.get_width() // 2,
                20
            )
        )

        score_surf = self.font_medium.render(
            f"Score: {self.score}",
            True,
            (255, 220, 80)
        )
        screen.blit(
            score_surf,
            (
                self.width // 2 - score_surf.get_width() // 2,
                60
            )
        )

        status_text = (
            "Watch the pattern..."
            if self.state == "WATCH"
            else "Your turn: Click the pattern!"
        )

        status_color = (

```

```

        (190, 195, 205)
        if self.state == "WATCH"
        else (80, 240, 130)
    )

    if self.state == "GAME_OVER":
        status_text = "Game Over - Press R to restart"
        status_color = (240, 70, 70)

    status_surf = self.font_medium.render(
        status_text,
        True,
        status_color
    )

    screen.blit(
        status_surf,
        (
            self.width // 2 - status_surf.get_width() // 2,
            95
        )
    )

    for btn in self.buttons:
        btn.render(screen)

    # Countdown bar during player's turn.
    if self.state == "PLAYER_TURN":
        elapsed = pygame.time.get_ticks() -
self.player_turn_start
        remaining = max(
            0,
            self.player_turn_duration - elapsed
        )

        progress = remaining / self.player_turn_duration

        bar_width = 300
        bar_height = 18
        bar_x = self.width // 2 - bar_width // 2
        bar_y = 430

```

```

pygame.draw.rect(
    screen,
    (60, 65, 75),
    (bar_x, bar_y, bar_width, bar_height),
    border_radius=8
)

pygame.draw.rect(
    screen,
    (80, 200, 120),
    (
        bar_x,
        bar_y,
        int(bar_width * progress),
        bar_height
    ),
    border_radius=8
)

timer_text = self.font_medium.render(
    f"Time: {remaining / 1000:.1f}s",
    True,
    (230, 230, 230)
)

screen.blit(
    timer_text,
    (
        self.width // 2 - timer_text.get_width() // 2,
        455
    )
)

if self.state == "GAME_OVER":
    overlay = pygame.Surface(
        (self.width, self.height),
        pygame.SRCALPHA
    )
    overlay.fill((0, 0, 0, 200))
    screen.blit(overlay, (0, 0))

```

```

over_surf = self.font_title.render(
    "WRONG PATTERN! GAME OVER",
    True,
    (240, 70, 70)
)

screen.blit(
    over_surf,
    (
        self.width // 2 - over_surf.get_width() // 2,
        self.height // 2 - 40
    )
)

final_score_surf = self.font_medium.render(
    f"Final Score: {self.score}",
    True,
    (255, 255, 255)
)

screen.blit(
    final_score_surf,
    (
        self.width // 2 - final_score_surf.get_width()
// 2,
        self.height // 2 + 10
    )
)

restart_surf = self.font_medium.render(
    "Press [R] to Play Again",
    True,
    (200, 200, 200)
)

screen.blit(
    restart_surf,
    (
        self.width // 2 - restart_surf.get_width() // 2,
        self.height // 2 + 50
    )
)

```

```
)  
)
```

Requirements:

- Each round appends exactly one new random color.
- Flash duration decreases with score, minimum 180 ms.
- Pause duration decreases with score, minimum 80 ms.
- Red = 261 Hz.
- Blue = 329 Hz.
- Green = 392 Hz.
- Yellow = 523 Hz.
- Sounds play during WATCH and PLAYER_TURN clicks.
- A visible countdown bar is shown during PLAYER_TURN.
- Timer expiration changes PLAYER_TURN to GAME_OVER.
- Incorrect player input still immediately causes GAME_OVER.
- R still resets the game correctly.

9/30/2026, 12:16:03 PM



Your final implementation satisfies the listed requirements. I found **no missing requirement that needs a code change**.

Requirement-by-requirement review

Requirement	Status	Relevant code
Exactly one new random color per round	✓	<code>self.sequence.append(new_color)</code>
Flash duration decreases with score	✓	<code>450 - self.score * 20</code>
Flash minimum = 180 ms	✓	<code>max(180, ...)</code>
Pause duration decreases with score	✓	<code>200 - self.score * 10</code>

Requirement	Status	Relevant code
Pause minimum = 80 ms	✓	<code>max(80, ...)</code>
Red = 261 Hz	✓	<code>0: 261</code>
Blue = 329 Hz	✓	<code>1: 329</code>
Green = 392 Hz	✓	<code>2: 392</code>
Yellow = 523 Hz	✓	<code>3: 523</code>
Sound during WATCH	✓	<code>play_color_sound()</code> when first/next button lights
Sound during PLAYER_TURN clicks	✓	<code>self.play_color_sound(btn.color_id)</code>
Visible countdown bar	✓	<code>pygame.draw.rect(...)</code> under <code>PLAYER_TURN</code>
Timer updates with elapsed time	✓	<code>remaining = ... - elapsed</code>
Timer expiration → GAME_OVER	✓	<code>if elapsed >= self.player_turn_duration</code>
Incorrect input → immediate GAME_OVER	✓	Existing comparison remains unchanged
R resets game	✓	<code>GAME_OVER</code> → <code>reset()</code> → <code>start_next_round()</code>

One small implementation detail to be aware of

Your timer is **5 seconds**, because you have:

```
self.player_turn_duration = 5000
```

That's perfectly valid based on the requirements you provided; they specify a limited amount of time but don't prescribe a particular duration.

Reset behavior

Your reset flow is also correct:

```
R
↓
reset()
↓
sequence.clear()
player_input.clear()
score = 0
↓
start_next_round()
↓
new sequence of length 1
↓
WATCH
```

And because `start_player_turn()` sets:

```
self.player_turn_start = pygame.time.get_ticks()
```

each new round gets a fresh 5-second timer.

One thing that is *not* a bug

You use:

```
self.flash_duration = max(180, 450 - self.score * 20)
self.pause_duration = max(80, 200 - self.score * 10)
```

rather than the earlier example of `15/8`. That's fine. The requirements only say the durations should **gradually decrease with score** and respect the minimums. Your implementation does that.

Also, your sound initialization gracefully handles an unavailable mixer:

```
except (pygame.error, ValueError):
    self.sounds = {}
```

So the game can continue without audio rather than crashing. That doesn't interfere with the requested behavior when Pygame audio is available.

Conclusion: based specifically on the four tasks and the requirements you listed, I would consider this implementation complete. No unrelated changes are necessary.

9/30/2026, 12:16:06 PM · gpt-5-6